

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze

10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate
----	--	-----------------------------

Code of Conduct:

I __Shaik Ayesha Tabassum(192525074)____ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Sequential File Organization vs Heap File Organization

Feature	Sequential File Organization	Heap File Organization
Data insertion	New records must be placed in the correct sorted/sequence position. Insertion can be slow because records may need to be shifted or overflow areas used.	Very fast. New records are usually inserted at the end or in any available free space.
Data deletion	Deletion can be slower because the sequence/order must be maintained. Deleted spaces may need to be reorganized later.	Easy and fast. The record can simply be marked deleted or its space reused.
Retrieval	Very efficient for sequential access and range queries because records are ordered. Binary search can also be used for exact searches.	Slow for searching a particular record because the system may need to scan the entire file.
Exact search	Faster, especially when an index or binary search is available.	Usually requires a linear search: O(n) .
Range search	Very efficient because related records are stored together.	Inefficient because the entire file may need to be scanned.
Space utilization	May have unused/overflow space due to insertions and deletions.	Generally simple and flexible space utilization.
Best suited for	Applications requiring frequent searches, ordered processing, and range queries.	Applications with frequent insertions and where fast insertion is more important than search speed.

Example

Suppose a student file contains records sorted by **Student ID**:

Sequential:

101 → 105 → 110 → 115 → 120

If a student with ID **108** is inserted, the file must maintain the order:

101 → 105 → 108 → 110 → 115 → 120

This makes insertion more expensive.

Heap:

Records can be stored in any order:

110 → 101 → 120 → 105 → 115

A new record can simply be added at the end, making insertion very fast.

Conclusion

Sequential file organization provides better retrieval efficiency, especially for ordered and range-based queries, but insertion and deletion are relatively costly.

Heap file organization provides very fast insertion and simple deletion, but retrieval is slower because records are not ordered.

2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Clustered Indexing vs Non-Clustered Indexing

Feature	Clustered Indexing	Non-Clustered Indexing
Meaning	Determines the physical order in which records are stored in the table.	Creates a separate index structure that points to the actual records.
Data order	Table data is physically arranged according to the index key.	Table data does not need to be physically arranged according to the index key.
Number per table	Usually one clustered index because data can have only one physical order.	Multiple non-clustered indexes can be created on the same table.
Search performance	Very fast for range searches and ordered retrieval.	Fast for searches on the indexed column.
Insertion	Can be slower because records may need to be inserted in the correct physical position.	Usually easier because the table's physical order does not change.
Storage	Index is closely associated with the table data.	Requires additional storage for the separate index structure.
Example	Index on student_id where student records are physically ordered by student_id.	Index on student_name that contains pointers to the corresponding student records.

Example

Consider a Student table:

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(20),  
    marks INT  
);
```

1. Clustered Index

Suppose the records are physically arranged according to student_id:

```
101 Ayesha CSE 85  
102 Rahul ECE 78  
103 Priya CSE 91  
104 Arjun IT 82
```

A clustered index on student_id allows the database to retrieve records efficiently in ID order.

For example:

```
SELECT * FROM Student  
WHERE student_id BETWEEN 101 AND 103;
```

This is efficient because the records are stored in the same order as the index.

2. Non-Clustered Index

Suppose we create an index on name:

```
CREATE INDEX idx_student_name  
ON Student(name);
```

The index maintains names separately and points to the corresponding records:

Index
Ayesha → Record 101
Arjun → Record 104
Priya → Record 103
Rahul → Record 102

The actual table does not have to be physically sorted by name.

A query such as:

```
SELECT * FROM Student  
WHERE name = 'Priya';
```

can use the non-clustered index to locate the required record quickly.

Simple Difference

Clustered indexing: Data itself is stored in the index order.

Non-clustered indexing: Index is stored separately and contains references to the actual data.

Example:

student_id → Clustered index
student_name → Non-clustered index


```
mysql> SHOW INDEX FROM Student;
```

```
mysql> SELECT *
```

```

+-----+-----+-----+-----+-----+
| student_id | name | department | marks | city |
+-----+-----+-----+-----+-----+
|          103 | Priya | CSE        | 92    | Bangalore |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Student	NULL	ref	idx_student_name	idx_student_name	203	const	1	100.00	NULL

1 row in set, 1 warning (0.00 sec)

3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

Primary Indexing vs Secondary Indexing

Feature	Primary Indexing	Secondary Indexing
Meaning	Index created on a primary key or ordering key of a sorted file.	Index created on a non-ordering attribute.
Data ordering	Data records are physically ordered according to the indexed field.	Data records are not physically ordered according to the indexed field.
Storage requirement	Requires less storage , especially when a sparse index is used.	Requires more storage because a dense index is generally needed.
Search performance	Very fast because the data is ordered by the index field.	Fast for searches, but generally requires an additional pointer/reference to locate records.
Number of indexes	Usually one primary index for a file.	Multiple secondary indexes can be created.
Duplicate values	Usually uses a unique field such as Student ID.	Can be created on fields with duplicate values such as Department.
Example	Index on student_id.	Index on department or student_name.

Example

Consider a Student table:

Student_ID	Name	Department
101	Ayesha	CSE
102	Rahul	ECE
103	Priya	CSE
104	Arjun	IT

Primary Indexing

If the file is physically sorted by Student_ID, a primary index can be created:

101 → Block 1
103 → Block 2

Since the records are ordered, searching for a particular student ID is efficient.

For example:

```
SELECT * FROM Student
WHERE Student_ID = 103;
```

The index quickly identifies the block containing the record.

Secondary Indexing

Suppose we create a secondary index on Department:

```
CREATE INDEX idx_department  
ON Student(Department);
```

The index may contain:

CSE → 101, 103

ECE → 102

IT → 104

This requires additional storage, particularly when many records have the same value.

Storage and Performance Comparison

- **Primary indexing:** Lower storage requirement and very efficient search because the data is already ordered.
- **Secondary indexing:** Higher storage requirement because additional index entries and record pointers are needed, but it allows efficient searching on fields that are not used for physical ordering.

Conclusion

Primary indexing is more storage-efficient and provides fast ordered searches, while secondary indexing uses more storage but provides flexible and fast searches on non-ordering attributes.

4. Analyze the differences between static hashing and dynamic hashing in database systems.

Static Hashing vs Dynamic Hashing

Hashing is a file organization technique that uses a **hash function** to determine where a record should be stored.

Feature	Static Hashing	Dynamic Hashing
Meaning	Number of buckets is fixed when the file is created.	Number of buckets can grow or shrink as data changes.
Bucket size	Fixed.	Can be dynamically adjusted.
Growth	Difficult to handle when the file grows.	Handles file growth efficiently.
Overflow	Overflow buckets are commonly required when buckets become full.	Reduces overflow using bucket splitting or directory expansion.
Performance	Good when the number of records is close to the expected size.	Maintains good performance even when the number of records changes significantly.
Storage utilization	May waste space if too many buckets are allocated initially.	Generally uses storage more efficiently.
Implementation	Simple to implement.	More complex to implement.
Insertion	Becomes slower when many overflow buckets are created.	Usually remains efficient because new buckets can be created.
Search	Normally O(1) average time, but overflow can increase search cost.	Also approximately O(1) average time with proper bucket management.
Best suited for	Databases with relatively stable data size.	Databases where the amount of data changes frequently.

1. Static Hashing

In static hashing, the number of buckets is fixed.

For example, suppose there are **4 buckets**:

Hash Function: $h(\text{key}) = \text{key} \% 4$

Bucket 0

Bucket 1

Bucket 2

Bucket 3

For Student_ID = 101:

$$101 \% 4 = 1$$

So the record is stored in **Bucket 1**.

If Bucket 1 becomes full, an **overflow bucket** may be created:

Bucket 1 → Overflow Bucket

Problem: If the database grows significantly, many overflow buckets may be created, increasing search time.

2. Dynamic Hashing

Dynamic hashing allows the number of buckets to change according to the amount of data.

Two common techniques are:

- **Extendible hashing**
- **Linear hashing**

For example, in **extendible hashing**, when a bucket becomes full, it can be split. If necessary, the directory is doubled.

Before:

Directory

0 → Bucket A

1 → Bucket B

After overflow:

Directory

00 → Bucket A

01 → Bucket C

10 → Bucket B

11 → Bucket D

This allows the database to accommodate more records without depending heavily on overflow buckets.

Analysis

Static hashing is simpler and works well when the database size is predictable. However, if the database grows, overflow chains can increase and degrade search performance.

Dynamic hashing is more flexible because buckets can expand or split as data grows. It provides more consistent performance but requires additional processing and a more complex structure.

Conclusion

Static hashing = Fixed buckets + Simple implementation + Overflow problems

Dynamic hashing = Flexible buckets + Better handling of growth + More complex implementation

For a **small and relatively stable database**, static hashing is suitable. For a **large database with frequent insertions and changing data size**, dynamic hashing is generally more efficient.

5. Compare B-Tree indexing and B+ Tree indexing for large- scale database applications.

B-Tree vs B+ Tree Indexing

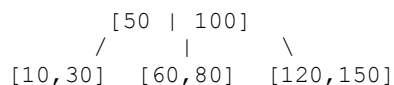
Both **B-Tree** and **B+ Tree** are balanced tree indexing techniques used in database systems to efficiently search large amounts of data.

Feature	B-Tree	B+ Tree
Data storage	Data records/pointers can be stored in both internal and leaf nodes.	Actual data pointers are stored only in leaf nodes.
Internal nodes	Store keys and data pointers.	Store only keys and pointers to child nodes.
Leaf nodes	May not be linked.	Leaf nodes are linked sequentially.
Search	Fast because data can be found at internal or leaf nodes.	Very efficient and predictable because searches reach leaf nodes.
Range queries	Less efficient.	Highly efficient because leaf nodes are linked.
Fan-out	Lower because internal nodes store data pointers.	Higher because internal nodes store only keys and child pointers.
Tree height	Can be slightly higher.	Usually shorter for large datasets.
Sequential access	Less convenient.	Very efficient through linked leaf nodes.
Storage efficiency	Lower for indexes because internal nodes contain data pointers.	Higher because internal nodes contain only keys and pointers.
Database usage	Less commonly preferred for database indexing.	Widely used in database systems.

Example

Suppose a database contains millions of student records indexed by student_id.

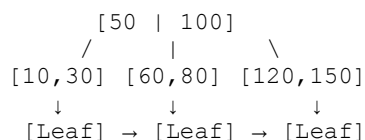
B-Tree



Data pointers can exist at both internal and leaf nodes.

If we search for 80, the record may be found directly in an internal/leaf node.

B+ Tree



Therefore, a range query such as:

can be processed efficiently by locating 60 and then following the linked leaf nodes.

For large-scale databases, B+ Trees generally provide better performance because:

- ## Conclusion

For **large-scale database applications**, **B+ Tree is generally preferred** because it provides higher fan-out, shorter trees, efficient range queries, and fast sequential access.

```
MySQL 8.0 Command Line Cli + - ×
```

```
mysql> CREATE INDEX idx_marks  
-> ON Student(marks);  
Query OK, 0 rows affected (0.04 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> EXPLAIN  
-> SELECT *  
-> FROM Student  
-> WHERE marks = 85;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Student	NULL	ref	idx_marks	idx_marks	5	const	1	100.00	NULL

```
1 row in set, 1 warning (0.00 sec)
```

```
mysql> EXPLAIN  
-> SELECT *  
-> FROM Student  
-> WHERE marks BETWEEN 75 AND 90;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Student	NULL	range	idx_marks	idx_marks	5	NULL	6	100.00	Using index condition

```
1 row in set, 1 warning (0.00 sec)
```

```
MySQL 8.0 Command Line Cli  x  +  v  -  [ ]  x

1 row in set, 1 warning (0.00 sec)

mysql> CREATE INDEX idx_marks
-> ON Student(marks);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> EXPLAIN
-> SELECT *
-> FROM Student
-> WHERE marks = 85;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | ref | idx_marks | idx_marks | 5 | const | 1 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Student
-> WHERE marks BETWEEN 75 AND 90;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | range | idx_marks | idx_marks | 5 | NULL | 6 | 100.00 | Using index condition |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Linear Hashing vs Extendible Hashing

Both **Linear Hashing** and **Extendible Hashing** are dynamic hashing techniques. They allow a hash file to grow as new records are inserted without requiring a completely new hash table.

Feature	Linear Hashing	Extendible Hashing
Bucket management	Buckets are split gradually in a predefined linear order.	Buckets are split based on hash values and directory entries.
Directory	Does not require a directory .	Uses a directory containing pointers to buckets.
Bucket splitting	One bucket is split at a time according to a split pointer.	The bucket causing overflow is split.
Growth	Grows gradually and smoothly.	Directory may double when required.
Scalability	Good for continuously growing databases.	Very good for large databases with unpredictable growth.
Memory overhead	Lower because there is no directory.	Higher because of the directory.
Overflow handling	Overflow can occur temporarily until the bucket is split.	Bucket splitting reduces overflow quickly.
Implementation	Relatively simple.	More complex because directory management is required.
Performance	Good average performance with gradual expansion.	Very efficient search because the directory directly identifies the bucket.

1. Linear Hashing

Linear hashing uses a **split pointer** to determine which bucket should be split next.

For example:

Bucket 0
Bucket 1
Bucket 2
Bucket 3
↓
Split pointer

When a bucket needs expansion, buckets are split **one by one**:

Before:
B0 B1 B2 B3

After:

B0 B1 B2 B3 B4

↑
split gradually

It does not need to double a directory, so memory usage is relatively low.

2. Extendible Hashing

Extendible hashing uses a **directory** that points to buckets.

Directory

00 → Bucket A

01 → Bucket B

10 → Bucket C

11 → Bucket D

When a bucket overflows, it is split. If the local depth becomes greater than the global depth, the directory is doubled:

Before:

0 → A

1 → B

After:

00 → A

01 → C

10 → B

11 → D

This makes extendible hashing highly scalable for rapidly growing datasets.

Key Difference

Linear Hashing:

Grows **gradually**, one bucket at a time, without a directory.

Extendible Hashing:

Uses a **directory** and can expand the directory by doubling its size.

Conclusion

Linear hashing is preferable when gradual growth and lower memory overhead are important.
Extendible hashing is preferable when fast access and highly dynamic, unpredictable growth are important.

7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Dense Indexing vs Sparse Indexing

Both **dense indexing** and **sparse indexing** are used to speed up record retrieval in a database.

Feature	Dense Indexing	Sparse Indexing
Index entries	Contains an index entry for every record .	Contains index entries for some records , usually one per block.
Memory usage	Higher , because there are many index entries.	Lower , because fewer entries are stored.
Access speed	Generally faster because every record has a direct index entry.	Slightly slower because after finding the nearest index entry, the block may need to be searched.
Search	Directly identifies the required record.	Identifies the appropriate block, followed by a sequential search within the block.
Storage requirement	Large.	Small.
Maintenance	More expensive because every record insertion/deletion may require index updates.	Easier to maintain because fewer index entries are updated.
Requirement	Can be used even when records are not ordered, depending on the index type.	Usually requires the data file to be sorted/ordered on the search key.

Example

Suppose a file contains 10,000 student records.

Dense Index

Every record has an index entry:

101 → Record 1

102 → Record 2

103 → Record 3

104 → Record 4

...

10000 → Record 10000

Therefore, there are approximately **10,000 index entries**.

Advantage: Fast access.

Disadvantage: High memory/storage requirement.

Sparse Index

Suppose each block contains 100 records. The index can contain one entry for each block:

101 → Block 1
201 → Block 2
301 → Block 3
401 → Block 4
...

For 10,000 records and 100 records per block, only about **100 index entries** are needed.

To find record 350:

1. Search the sparse index.
2. Find that 301 points to Block 3.
3. Search within Block 3 for record 350.

Advantage: Much lower memory usage.

Disadvantage: Slightly slower access compared with dense indexing.

Simple Comparison

Dense Index → More memory → Faster access
Sparse Index → Less memory → Slightly slower access

Conclusion

Dense indexing provides faster record access but requires more memory because every record has an index entry.

Sparse indexing saves memory by storing fewer index entries, but access can be slightly slower because the required block must be searched after locating the nearest index entry.

For large databases where memory efficiency is important, sparse indexing is often preferred when the data is sorted.

```

mysql> CREATE INDEX idx_marks
-> ON Student(marks);
ERROR 1061 (42000): Duplicate key name 'idx_marks'
mysql> SHOW INDEX FROM Student;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| student | 0 | PRIMARY | 1 | student_id | A | 10 | NULL | NULL | YES | |
| BTREE | | | YES | NULL | | | | | | |
| student | 1 | idx_student_name | 1 | name | A | 10 | NULL | NULL | YES |
| BTREE | | | YES | NULL | | | | | | |
| student | 1 | idx_department | 1 | department | A | 3 | NULL | NULL | YES |
| BTREE | | | YES | NULL | | | | | | |
| student | 1 | idx_marks | 1 | marks | A | 10 | NULL | NULL | YES |
| BTREE | | | YES | NULL | | | | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
mysql> EXPLAIN
-> SELECT *
-> FROM Student
-> WHERE marks = 88;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | ref | idx_marks | idx_marks | 5 | const | 1 | 100.00 | NULL |

```

8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Indexed Sequential File Organization vs Direct File Organization

Both are file organization techniques used to store and retrieve records, but they differ in how records are accessed.

Feature	Indexed Sequential File Organization	Direct File Organization
Storage method	Records are stored sequentially and an index is maintained.	Records are placed directly into locations using a hash function or address calculation.
Sequential access	Very efficient because records are ordered.	Less efficient for sequential processing.
Direct access	Fast because the index identifies the required block.	Very fast because the hash/address directly identifies the location.
Range queries	Excellent because records are stored in order.	Poorer because related records may be distributed across different locations.
Insertion	Can be slower because records must maintain sequence; overflow areas may be needed.	Generally fast, although collisions may require overflow handling.
Deletion	May require index updates and management of empty spaces.	Usually simple, but deleted locations may need to be managed.
Storage overhead	Higher because both data and index are stored.	Lower index overhead because a separate index is generally not required.
Scalability	Good, but index and overflow areas need management as data grows.	Good for large datasets with predictable hash distribution.
Best suited for	Applications requiring both sequential and random access.	Applications requiring frequent exact-key searches.

Indexed Sequential File Organization

In this method, records are stored in sorted order and an index is maintained.

Index

101 → Block 1

201 → Block 2

301 → Block 3

Data

Block 1: 101, 120, 150

Block 2: 201, 225, 280

Block 3: 301, 340, 390

Advantages

1. **Supports both sequential and direct access.**
2. **Fast searching** using the index.
3. **Efficient range queries** because records are ordered.
4. Suitable for applications such as payroll and banking systems where records may need to be processed sequentially.

Limitations

1. Requires **additional storage for the index.**
2. Insertions and deletions can be more expensive.
3. Overflow blocks may occur when new records cannot be inserted in their proper position.
4. Index maintenance is required when the file changes.

Direct File Organization

In direct organization, a hash function or address calculation determines where a record is stored.

For example:

Hash function:

$$h(\text{Student_ID}) = \text{Student_ID} \% 10$$

For Student_ID = 105:

$$105 \% 10 = 5$$

So the record is placed in **Bucket 5**.

Advantages

1. **Very fast direct access** for a known key.
2. No separate index is normally required.
3. Efficient for frequent insertions and exact searches.
4. Suitable for applications such as bank account lookup and customer record retrieval.

Limitations

1. **Poor for range queries.**
2. Sequential processing is less efficient.
3. **Hash collisions** can occur.
4. Overflow buckets or collision-resolution techniques may be required.
5. Performance depends on a good hash function and appropriate bucket distribution.

Overall Analysis

Indexed sequential organization is more flexible because it supports both sequential and direct access and is particularly good for range queries.

Direct organization provides faster exact-key access and simpler retrieval, but it is not suitable when records must frequently be processed in sorted order or queried by ranges.

Conclusion

Indexed Sequential → Best for sequential + range + direct access

Direct → Best for fast exact-key access

For a **banking or payroll system** requiring reports and range-based processing, indexed sequential organization is suitable. For a **customer lookup system** where records are mainly searched using a unique ID, direct organization is generally more efficient.

9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Single-Level Indexing vs Multi-Level Indexing

Indexing improves database retrieval by reducing the number of disk blocks that need to be searched.

Feature	Single-Level Indexing	Multi-Level Indexing
Meaning	Uses one index level to locate records or blocks.	Uses multiple levels of indexes, where one index points to another index.
Structure	Simple and flat.	Hierarchical, similar to a tree.
Search speed	Fast for small indexes.	Faster for very large databases because the search space is reduced at each level.
Memory usage	Requires more memory if the index becomes very large.	More efficient for large indexes because higher-level indexes contain fewer entries.
Disk I/O	Can increase when the index is large.	Reduces disk I/O by narrowing the search at each level.
Implementation	Simple.	More complex.
Scalability	Suitable for small and medium-sized files.	Highly suitable for large-scale databases.
Maintenance	Easy to maintain.	More complex because changes may affect multiple levels.

1. Single-Level Indexing

A single-level index contains one index structure pointing directly to data blocks.

Index

101 → Block 1

201 → Block 2

301 → Block 3

401 → Block 4

↓

Data

To search for student ID 301, the database searches the index and directly accesses **Block 3**.

Advantages:

- Simple to implement.
- Fast when the index is small.
- Easy to maintain.

Limitation:

If the database contains millions of records, the index itself can become very large, requiring more memory and disk accesses.

2. Multi-Level Indexing

In multi-level indexing, an index is created on the first-level index.

Level 2 Index



Level 1 Index



Data Blocks

For example:

Level 2:

1 → Index Block A

2 → Index Block B

Level 1:

101 → Data Block 1

201 → Data Block 2

301 → Data Block 3

401 → Data Block 4

The database first searches **Level 2**, then the appropriate **Level 1 index**, and finally the data block.

Retrieval Comparison

Suppose a database contains **1 million records**.

Single-level indexing:

Search large index → Data block

Multi-level indexing:

Level 2 → Level 1 → Data block

Although multi-level indexing involves more levels, each level contains fewer entries. Therefore, the database can locate the required data with fewer overall disk accesses for a very large index.

Conclusion

Single-level indexing is simple and effective for smaller databases.

Multi-level indexing is more complex but provides better scalability and retrieval performance for very large databases.

Single-Level → Simple + Easy to maintain

Multi-Level → Scalable + Efficient for large databases

10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Hashing vs Indexing for Exact-Match and Range Queries

Hashing and indexing are two important techniques used to improve database retrieval performance. Their effectiveness depends on the type of query being performed.

Feature	Hashing	Indexing
Exact-match query	Excellent	Very good
Range query	Poor	Excellent
Search complexity	Average O(1)	Generally O(log n) with B+ Tree
Data ordering	Not ordered	B+ Tree maintains ordered keys
Sequential access	Poor	Good
Storage overhead	Buckets and overflow space	Index structure requires additional storage
Best suited for	Exact-key searches	Exact searches and range searches
Example	WHERE student_id = 105	WHERE marks BETWEEN 70 AND 90

1. Hashing for Exact-Match Queries

Hashing uses a hash function to calculate the location of a record.

For example:

$$h(\text{student_id}) = \text{student_id} \% 10$$

For:

$$\begin{aligned}\text{student_id} &= 105 \\ 105 \% 10 &= 5\end{aligned}$$

The database can directly access **Bucket 5**.

Therefore, hashing provides approximately **O(1) average search time** for exact-match queries.

Example:

```
SELECT *  
FROM Student  
WHERE student_id = 105;
```

Performance: Excellent.

2. Hashing for Range Queries

Consider:

```
SELECT *  
FROM Student  
WHERE student_id BETWEEN 100 AND 150;
```

Hashing is inefficient because records with IDs from 100 to 150 may be distributed across many unrelated buckets.

The database may need to examine many buckets.

Performance: Poor.

3. Indexing for Exact-Match Queries

A B+ Tree index can efficiently locate a specific record.

Example:

```
SELECT *  
FROM Student  
WHERE student_id = 105;
```

The database traverses the B+ Tree from the root to the appropriate leaf.

Performance: Approximately $O(\log n)$.

Although this is theoretically slower than average-case hashing, B+ Tree indexing is highly efficient and provides additional benefits such as ordered and sequential access.

4. Indexing for Range Queries

Indexing is especially effective for range queries.

Example:

```
SELECT *  
FROM Student  
WHERE marks BETWEEN 70 AND 90;
```

A B+ Tree can:

1. Locate the first value (70).
2. Follow the linked leaf nodes.
3. Retrieve values until 90 is reached.

Therefore, range retrieval is highly efficient.

Performance: Excellent.

Performance Summary

	Exact Match	Range Query
Hashing	Excellent	Poor
B+ Tree Index	Very Good	Excellent

Practical Example

Consider a **banking database**:

Find one account using account number:

```
SELECT *  
FROM Account  
WHERE account_no = 50025;
```

Hashing is highly suitable because the query is an exact-match search.

For finding accounts with balances between ₹50,000 and ₹1,00,000:

```
SELECT *  
FROM Account  
WHERE balance BETWEEN 50000 AND 100000;
```

A **B+ Tree index** is much more suitable because it supports efficient range retrieval.

Conclusion

- **Hashing** → **Best for exact-match queries**
- **B+ Tree indexing** → **Best for range queries**
- **B+ Tree indexing** → **Also provides efficient exact-match searches**
- **Hashing** → **Generally unsuitable for range queries**

Therefore, database designers should choose the technique based on the workload. **Hashing is preferred for frequent equality searches, while B+ Tree indexing is preferred when both equality and range queries are important.**

```
MySQL 8.0 Command Line Cli
mysql> EXPLAIN
-> SELECT *
-> FROM Student
-> WHERE student_id = 105;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | const | PRIMARY | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Student
-> WHERE student_id BETWEEN 103 AND 108;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | range | PRIMARY | PRIMARY | 4 | NULL | 6 | 100.00 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

```
mysql> SELECT *
-> FROM Student
-> WHERE student_id BETWEEN 103 AND 108;
+-----+-----+-----+-----+-----+-----+
| student_id | name | department | marks | city |
+-----+-----+-----+-----+-----+-----+
| 103 | Priya | CSE | 92 | Bangalore |
| 104 | Arjun | IT | 74 | Chennai |
| 105 | Sneha | CSE | 88 | Mumbai |
| 106 | Kiran | ECE | 69 | Delhi |
| 107 | Anjali | IT | 95 | Chennai |
| 108 | Ravi | CSE | 81 | Hyderabad |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```